



**Kampus
Merdeka**
INDONESIA JAYA

MODUL PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

S1-INFORMATIKA

Disusun Oleh :

TIM DOSEN

Praktikum 11

Abstract Class dan Interface

Tujuan Praktikum :

1. Mahasiswa mampu membuat kode Java berbasis OOP.
2. Mahasiswa mampu mengimplementasikan konsep Abstract Class dan Interface.
3. Mahasiswa mampu menerapkan konsep dasar, mendeklarasikan, menggunakan, dan mewariskan interface.

Alat dan Bahan :

1. IntelliJ Idea

Materi :

1. Abstract Class

Abstract class atau kelas abstrak adalah kelas yang terletak di posisi tertinggi dalam hierarki *class*. *Class* ini tidak dapat diinstansiasi karena masih bersifat abstrak. *Class* ini hanya berisi variabel umum dan deklarasi *method* tanpa detail penggunaannya (*abstract method*). Selanjutnya *class* yang menjadi turunan dari *abstract class* ini yang akan menentukan detail penggunaan *method* tersebut.

Deklarasi dan penggunaan *abstract class*.

Abstract Class dideklarasikan dengan cara sebagai berikut:

```
public abstract class NamaKelasAbstrak {  
    //deklarasi variabel dan abstract method  
    //definisi method tidak abstrak  
}
```

Abstract Class digunakan dengan cara sebagai berikut:

```
class NamaKelasPengguna extends NamaKelasAbstrak {  
    //penggunaan variabel dan method tidak abstrak  
    //pendefinisian abstract method  
}
```



2. Interface

Interface merupakan suatu mekanisme dalam Java yang memungkinkan untuk berbagi konstanta atau menentukan bentuk method yang dapat digunakan oleh sejumlah class. Sebuah class dapat mengimplementasikan lebih dari satu interface. Di dalam *interface*, penentu akses untuk definisi konstanta adalah *public static final*. Sedangkan penentu akses untuk deklarasi *abstract method* adalah *public abstract*. Kedua penentu akses ini tidak harus dituliskan secara eksplisit dalam kode program karena Java akan menggunakan kedua penentu akses ini sebagai penentu akses *default* bila sekiranya tidak ditulis secara eksplisit.

Deklarasi, penggunaan, dan pewarisan *interface*. *Interface* dideklarasikan dengan cara sebagai berikut:

```
public interface NamaInterface {  
    //definisi konstanta  
    //deklarasi abstract method  
}
```

Interface digunakan dengan cara sebagai berikut:

```
class NamaKelasPengguna implements NamaInterface {  
    //penggunaan konstanta  
    //pendefinisian abstract method  
}
```

Interface diwariskan dengan cara sebagai berikut:

```
public interface Interface_Y extends Interface_X {  
    //definisi konstanta  
    //deklarasi abstract method  
}
```

Perbedaan antara *Abstract Class* dan *Interface*

Saat deklarasi *abstract class* dan *interface* digunakan modifier *public*. Hal ini bertujuan agar *abstract class* dan *interface* tersebut dapat digunakan secara bebas oleh class lain yang membutuhkannya. Sekilas *interface* tampak mirip dengan



abstract class karena *abstract class* juga menentukan bentuk *method* untuk *subclass*, tetapi ada beberapa perbedaan antara *interface* dan *abstract class*, yaitu:

- a. *Abstract class* dapat mengandung *abstract method* maupun *method* tidak abstrak. Sedangkan *interface* hanya boleh mengandung *abstract method*.
- b. *Abstract class* dapat mendeklarasikan variabel *instant*. Sedangkan *interface* hanya dapat mendeklarasikan konstanta.
- c. *Abstract class* digunakan oleh *class* lain melalui pewarisan dengan kata kunci *extends*. Sedangkan sebuah *interface* diimplementasikan ke dalam suatu *class* dengan menggunakan kata kunci *implements*

Langkah Praktikum :

1. Studi Kasus : Hewan

Buat *abstract class* hewan dan kelas kucing, anjing yang mewarisi kelas hewan.

```
1  abstract class Hewan {
2      // Abstract method (tidak memiliki implementasi)
3      public abstract void suara();
4
5      // Non-abstract method (memiliki implementasi)
6      public void tidur() {
7          System.out.println(x:"Hewan sedang tidur...");
8      }
9  }
```

```
1  // Kelas Kucing yang meng-extend abstract class Hewan
2  class Kucing extends Hewan {
3      // Mengimplementasikan metode abstrak suara()
4      public void suara() {
5          System.out.println(x:"Kucing: Meow");
6      }
7  }
```



```

1 // Kelas Anjing yang meng-extend abstract class Hewan
2 class Anjing extends Hewan {
3     // Mengimplementasikan metode abstrak suara()
4     public void suara() {
5         System.out.println(x:"Anjing: Guk Guk");
6     }
7 }

```

```

1 // Main class untuk menjalankan program
2 public class MainHewan {
3     Run | Debug
4     public static void main(String[] args) {
5         Hewan kucing = new Kucing();
6         Hewan anjing = new Anjing();
7
8         kucing.suara(); // Output: Kucing: Meow
9         anjing.suara(); // Output: Anjing: Guk Guk
10        kucing.tidur(); // Output: Hewan sedang tidur...
11    }
12 }

```

2. Studi Kasus : Luas

Buat abstract class bentuk dan kelas lingkaran, persegi yang mewarisi kelas bentuk.

```

1 // Abstract class Bentuk
2 abstract class Bentuk {
3     // Abstract method untuk menghitung luas
4     public abstract double hitungLuas();
5
6     // Non-abstract method untuk menampilkan luas
7     public void tampilkanLuas() {
8         System.out.println("Luas bentuk adalah: " + hitungLuas());
9     }
10 }

```



```

1 // Kelas Lingkaran yang meng-extend abstract class Bentuk
2 class Lingkaran extends Bentuk {
3     private double jariJari;
4
5     public Lingkaran(double jariJari) {
6         this.jariJari = jariJari;
7     }
8
9     // Mengimplementasikan metode hitungLuas()
10    public double hitungLuas() {
11        return Math.PI * jariJari * jariJari;
12    }
13 }

```

```

1 // Kelas Persegi yang meng-extend abstract class Bentuk
2 class Persegi extends Bentuk {
3     private double sisi;
4
5     public Persegi(double sisi) {
6         this.sisi = sisi;
7     }
8
9     // Mengimplementasikan metode hitungLuas()
10    public double hitungLuas() {
11        return sisi * sisi;
12    }
13 }

```

```

1 // Main class untuk menjalankan program
2 public class MainLuas {
3     Run | Debug
4     public static void main(String[] args) {
5         Bentuk persegi = new Persegi(sisi:4);
6         Bentuk lingkaran = new Lingkaran(jariJari:3);
7
8         persegi.tampilkanLuas();
9         lingkaran.tampilkanLuas();
10    }
}

```

3. Studi Kasus : Kendaraan

Buat abstract class kendaraan dan kelas mobil, sepeda yang mewarisi kelas kendaraan.



```

1 // Abstract class Kendaraan
2 abstract class Kendaraan {
3     // Abstract method untuk mengembalikan kecepatan maksimum
4     public abstract int kecepatanMaksimum();
5
6     // Non-abstract method untuk menampilkan jenis kendaraan
7     public void tampilkanJenis() {
8         System.out.println(x:"Ini adalah kendaraan.");
9     }
10 }

```

```

1 // Kelas Mobil yang meng-extend abstract class Kendaraan
2 class Mobil extends Kendaraan {
3     // Mengimplementasikan metode kecepatanMaksimum()
4     public int kecepatanMaksimum() {
5         return 200; // kecepatan maksimum dalam km/jam
6     }
7 }

```

```

1 // Kelas Sepeda yang meng-extend abstract class Kendaraan
2 class Sepeda extends Kendaraan {
3     // Mengimplementasikan metode kecepatanMaksimum()
4     public int kecepatanMaksimum() {
5         return 30; // kecepatan maksimum dalam km/jam
6     }
7 }

```

```

1 // Main class untuk menjalankan program
2 public class MainKendaraan {
3     Run | Debug
4     public static void main(String[] args) {
5         Kendaraan mobil = new Mobil();
6         Kendaraan sepeda = new Sepeda();
7
8         mobil.tampilkanJenis();
9         System.out.println("Kecepatan maksimum mobil: " + mobil.kecepatanMaksimum() + " km/jam");
10
11         sepeda.tampilkanJenis();
12         System.out.println("Kecepatan maksimum sepeda: " + sepeda.kecepatanMaksimum() + " km/jam");
13     }
14 }

```

4. Studi Kasus : Pembayaran

Buat Interface pembayaran dan kelas kartu kredit, transfer bank mengimplementasikan interface kelas pembayaran.



```

1 // Interface Pembayaran
2 interface Pembayaran {
3     void prosesPembayaran(double jumlah);
4     double cekSaldo();
5 }

```

```

1 // Kelas KartuKredit yang mengimplementasikan interface Pembayaran
2 class KartuKredit implements Pembayaran {
3     private double limitSaldo;
4
5     public KartuKredit(double limitSaldo) {
6         this.limitSaldo = limitSaldo;
7     }
8
9     public void prosesPembayaran(double jumlah) {
10         if (jumlah <= limitSaldo) {
11             limitSaldo -= jumlah;
12             System.out.println("Pembayaran melalui Kartu Kredit sebesar " + jumlah + " berhasil.");
13         } else {
14             System.out.println(x:"Saldo tidak cukup di Kartu Kredit.");
15         }
16     }
17
18     public double cekSaldo() {
19         return limitSaldo;
20     }
21 }

```

```

1 // Kelas TransferBank yang mengimplementasikan interface Pembayaran
2 class TransferBank implements Pembayaran {
3     private double saldoRekening;
4
5     public TransferBank(double saldoRekening) {
6         this.saldoRekening = saldoRekening;
7     }
8
9     public void prosesPembayaran(double jumlah) {
10         if (jumlah <= saldoRekening) {
11             saldoRekening -= jumlah;
12             System.out.println("Pembayaran melalui Transfer Bank sebesar " + jumlah + " berhasil.");
13         } else {
14             System.out.println(x:"Saldo tidak cukup di rekening.");
15         }
16     }
17
18     public double cekSaldo() {
19         return saldoRekening;
20     }
21 }

```




```

1 // Main class untuk menjalankan program
2 public class MainPembayaran {
3     Run | Debug
4     public static void main(String[] args) {
5         Pembayaran kartuKredit = new KartuKredit(limitSaldo:5000000);
6         Pembayaran transferBank = new TransferBank(saldoRekening:3000000);
7
8         kartuKredit.prosesPembayaran(jumlah:2000000);
9         System.out.println("Saldo Kartu Kredit: " + kartuKredit.cekSaldo());
10
11        transferBank.prosesPembayaran(jumlah:1000000);
12        System.out.println("Saldo Rekening Bank: " + transferBank.cekSaldo());
13    }

```

5. Studi Kasus : Penerbang

```

1 // Interface Penerbang
2 interface Penerbang {
3     void terbang();
4 }

```

```

1 // Kelas Burung yang mengimplementasikan interface Penerbang
2 class Burung implements Penerbang {
3     public void terbang() {
4         System.out.println(x:"Burung terbang dengan sayapnya di udara.");
5     }
6 }

```

```

1 // Kelas Pesawat yang mengimplementasikan interface Penerbang
2 class Pesawat implements Penerbang {
3     public void terbang() {
4         System.out.println(x:"Pesawat terbang di langit dengan kecepatan tinggi.");
5     }
6 }

```



```

1  // Main class untuk menjalankan program
2  public class MainPenerbang {
3
4      Run | Debug
5      public static void main(String[] args) {
6          Penerbang pesawat = new Pesawat();
7          Penerbang burung = new Burung();
8
9          pesawat.terbang();
10         burung.terbang();
11     }
12 }

```

Lembar Kerja Praktik :

1. Buat kode program untuk menyelesaikan permasalahan sistem manajemen pekerja di sebuah perusahaan, yang berisi berbagai jenis pekerja dengan berbagai peran. Setiap pekerja memiliki cara berbeda dalam menyelesaikan tugas dan mendapatkan gaji. Dalam sistem ini, kita akan mengimplementasikan pekerja tetap dan pekerja lepas (freelancer) dengan metode yang berbeda untuk menghitung gaji mereka. Berikut ini detailnya :
 - a. Interface Tugas: Menyediakan metode untuk melaksanakan tugas dan menghitung gaji.
 - b. Abstract Class Pekerja: Menyimpan informasi umum tentang pekerja (seperti nama dan ID) dan mendeklarasikan metode abstrak untuk menghitung gaji.
 - c. Kelas PekerjaTetap dan PekerjaLepas: Mengimplementasikan Pekerja dengan logika spesifik untuk menghitung gaji mereka.
 - d. Kelas SistemManajemenPekerja: Mengelola pekerja dan tugas mereka.

Desain Sistem:

- a. Interface Tugas: Mendefinisikan metode untuk melaksanakan tugas dan menghitung gaji.
- b. Abstract class Pekerja: Berisi informasi umum tentang pekerja dan metode untuk menghitung gaji.



- c. Kelas PekerjaTetap: Menghitung gaji berdasarkan gaji tetap bulanan.
- d. Kelas PekerjaLepas: Menghitung gaji berdasarkan jumlah jam yang dikerjakan.

